

Project Description & Architectural Overview

Comprehensive description of the Enterprise Retrieval-Augmented Generation (RAG) and Code Generation Platform [cite: 1, 2].

Executive Overview: This project establishes a secure, modular, and high-availability Generative AI platform designed for enterprise environments [cite: 1, 2]. By bridging advanced internal document retrieval with automated code generation, the platform allows organizations to leverage large language models securely against private corpora without data leakage or hallucination vulnerabilities [cite: 1, 2].

Core Architecture & Functional Modules

The system is architected as a microservice framework divided into distinct functional tiers:

- **API Orchestration Layer (`main.py`):** Built using FastAPI and Pydantic, this tier delivers lightning-fast asynchronous endpoints for user queries and document ingestion with built-in validation [cite: 1, 2].
- **Knowledge Retrieval & RAG Pipeline (`rag\_engine.py`):** Implements recursive text chunking, Hugging Face transformer embeddings, and ChromaDB vector similarity search to ground LLM outputs in verified documents [cite: 1, 2].
- **Configuration & Security Management (`config.py`):** Handles parameter initialization and environment security through Pydantic Settings, keeping credentials completely isolated [cite: 1, 2].
- **Deployment & Containerization (`Dockerfile`):** Utilizes multi-stage Python builds to package the service into a secure, minimal-footprint container image ready for production environments [cite: 1, 2].

Operational Workflow

Data flows through the system in two primary phases: First, internal documents are ingested, split into optimized chunks, embedded via transformer models, and indexed in ChromaDB. Second, user prompts trigger a similarity search to retrieve relevant context blocks, which are subsequently assembled into prompts for accurate, context-grounded response generation [cite: 1, 2].